

Capstone Project: FinOps Forecasting Agent

Author: Claus Albuquerque

Course: Agentic AI Program: Building Autonomous Systems for Real-World Applications

Checkpoint: 1.1 - July 27, 2026.

Agent Type: Forecaster

The Agent, the Problem, and the Intended User

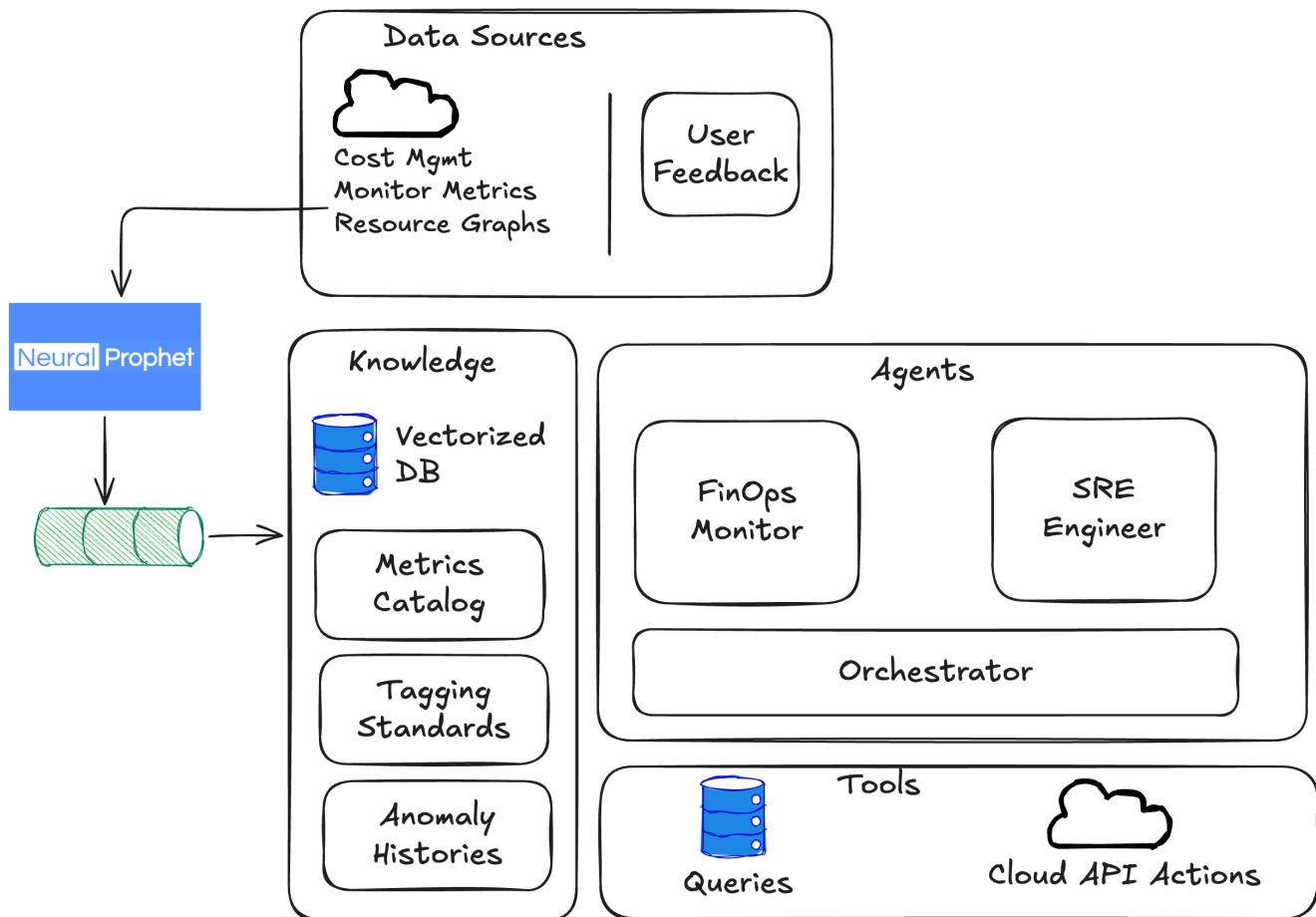
The FinOps Forecasting Agent is an intelligent system that continuously analyzes cloud cost and resource usage data to predict future spending, detect anomalies against forecasted baselines, and deliver actionable cost-optimization recommendations. The core problem it addresses is the inability of engineering and finance teams to anticipate cloud cost outcomes before they materialize on invoices. Cloud environments generate thousands of cost signals daily—across services, teams, regions, and environments—making it impractical for humans to manually track spending trajectories, identify deviations from expected patterns, or proactively surface savings opportunities before budgets are breached.

The intended users are FinOps engineers who need full-spectrum cost visibility, team leads who require budget-scoped forecasts for their services, and finance stakeholders who depend on reliable spend projections for planning. Each persona interacts with the agent differently: FinOps engineers ask granular questions and configure detection sensitivity; team leads consume proactive alerts and recommendations for their scope; finance stakeholders rely on forecasted budget-vs-actual projections.

Why a Standalone LLM or Simple Prompting Is Not Sufficient

A standalone LLM cannot forecast cloud costs because it lacks access to live billing data, resource utilization metrics, and historical baselines required to produce grounded predictions. Simple prompting would yield hallucinated cost figures with no connection to actual Cloud consumption. The agent must invoke structured tools—querying cost aggregates by service and time range, retrieving anomaly classifications, computing reservation coverage gaps—before it can reason about the data. Additionally, forecasting requires trained statistical and ML models (seasonal decomposition, Isolation Forest, Prophet) that produce numerical predictions the LLM alone is incapable of generating. The agent also enforces role-based access control, ensuring that a team lead cannot access another team's cost data through natural-language queries—a guardrail that pure prompting cannot implement. Finally, the system must continuously retrain its models using labeled feedback, which demands persistent state and scheduled computation beyond what a stateless prompt-response pattern supports.

The Environment



Data sources: Cloud Cost Management APIs (amortized and actual cost views), Cloud Monitor Metrics API (compute, storage, and network utilization per resource), Cloud Resource Graph (resource inventory snapshots), and a user feedback pipeline capturing alert responses and dashboard interactions.

Documents and knowledge: A metrics catalog defining formulas, thresholds, and ownership for every KPI; tagging standards (v1 and v2) that map resources to teams and cost centers; and anomaly summary histories stored as text for semantic retrieval.

Tools the agent invokes:

- *Read-only (autonomous):* `query_cost_by_service(service, date_range)` , `get_anomalies(severity, date_range)` , `get_ri_coverage()` , `forecast_spend(dimension, horizon)` , `get_idle_resources(threshold_days)` . These tools query PostgreSQL and the trained model registry to return structured, verified data.
- *Write actions (human-in-the-loop):* `downgrade_resource_sku(resource_id, target_sku)` , `deallocate_resource(resource_id)` , `delete_resource(resource_id)` . These tools require explicit human approval before execution—the agent proposes the action with estimated savings and impact analysis, the responsible owner reviews and confirms via an approval workflow (dashboard button or Slack prompt), and only then does the agent execute the change. Rejected proposals are logged as feedback to refine future recommendations.

Users: FinOps engineers (full access, configuration authority), team leads (team-scoped data), and finance stakeholders (organization-wide aggregates, read-only).

The Actions the Agent Takes

1. **Forecast:** Runs daily time-series predictions (Prophet-based) for each cost dimension, projecting next-day and end-of-month expected spend.
2. **Detect:** Compares actual costs against forecasted baselines using Z-score thresholds and Isolation Forest outputs; flags deviations as anomalies classified by type (spike, drift, drop, new-resource) and severity.
3. **Attribute and Route:** Assigns each anomaly to its root-cause dimension (resource, team, service, region) and routes alerts to the responsible owner via Slack, email, or ServiceNow.
4. **Recommend:** Generates ranked optimization suggestions—idle resource decommissioning, reservation purchases, rightsizing—with estimated dollar impact.
5. **Act (human-in-the-loop):** For approved recommendations, the agent executes write actions—downgrading a resource SKU, deallocating idle compute, or deleting orphaned resources. Each action follows a propose → review → confirm → execute workflow: the agent presents the change with estimated savings and blast-radius analysis, the responsible owner approves or rejects via a dashboard button or Slack prompt, and only upon explicit confirmation does the agent invoke the write tool. Rejections are captured as feedback signals.
6. **Converse:** Answers natural-language queries by retrieving relevant data through tool calls, augmenting the LLM context, and returning grounded, structured responses with supporting charts and action buttons.

How Feedback Guides Behavior Across Steps

Feedback operates as a closed loop across the entire pipeline. When a user labels an anomaly alert as a false positive, that label flows into the training dataset, causing the weekly model retraining to adjust thresholds for the affected dimension—reducing future noise. When a user dismisses a proactive recommendation card, the deduplication engine suppresses that recommendation type for the same resource, and the recommendation ranker down-weights similar suggestions.

Conversational responses collect thumbs-up/down ratings that feed into a weekly Agent Quality Report measuring groundedness and relevance; low-scoring query patterns trigger prompt refinements or tool-definition updates. Finally, dashboard interaction telemetry (filters used, drill-downs followed after an agent suggestion) provides implicit feedback on which recommendations actually drove user action, further refining the ranking algorithm. This multi-signal feedback loop enables the agent to improve forecast precision, reduce alert fatigue, and surface increasingly relevant recommendations over time.